# Exosuit Co-Contraction Cable Model

Cable-Driven 2R Planar Manipulator — Isaac Sim Implementation

Isaac Sim Robotics Project

April 2026

## Scope

This document describes the Isaac Sim port of the PyDrake exosuit co-contraction demo. It is the companion of `SEA_Cable_PyDrake_Implementation.tex` (drive cable only) and of the exo PyDrake document, and is the sibling of `Computed_Torque_IsaacSim_Implementation.tex`.

The target script is

`script_cup_manipulator_pendulam_tendon_with_exo_isaac_sim.py`

which is derived from

`script_cup_manipulator_pendulam_tendon_with_exo_pydrake.py`

by (i) replacing all Drake `LeafSystem` blocks with stateless NumPy callables, (ii) delegating rigid-body physics to Isaac Sim's PhysX `ArticulationView`, and (iii) porting cable + spring visualisation from Drake's Meshcat to USD cylinder prims.

- **New physics layer**: Antagonistic exosuit spring pair on joint 2 (pure unilateral extension springs, "pull-only").
- **Centred elbow pulley** (Method B): Both exo cables wrap on the *same* elbow pulley at radius $r_{\text{exo}}$, passing on opposite sides.
- **Co-contraction stiffness**: $k_{\text{eff}} = 2\,k_{\text{exo}}\,r_{\text{exo}}^2$ (at neutral elbow).
- **Activation dynamics**: First-order muscle-style ramp $\dot{a} = (a_{\text{cmd}} - a)/\tau_{\text{act}}$ with $\tau_{\text{act}} = 50\,\text{ms}$.
- **Isaac Sim integration**: `SEAExoActuatorNP` mirrors the Drake `SEAExoActuator` equations, called once per physics step.
- **Visual parity**: USD cylinder prims draw both drive and exo cables plus pulley wrap arcs, using a headless Drake plant to compute cable tangents.

- **Pipeline parity verified**: Circle trajectory with exo ON at $t = 2\,\text{s}$ produces identical $k_{\text{eff}} = 36.48\,\text{Nm/rad}$ on both simulators; elbow RMS tracking error within 0.6–1.3° on both.

## Contents

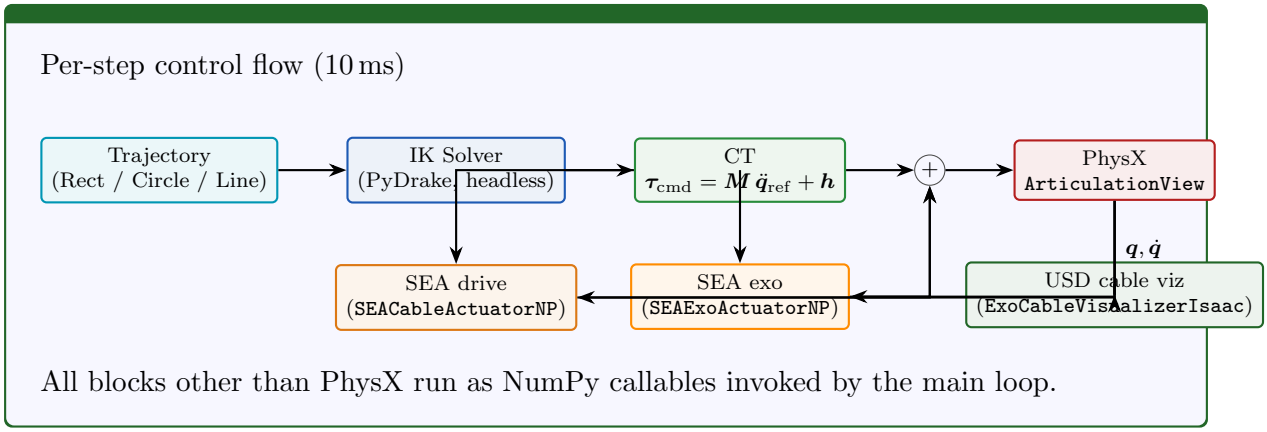
# 1 Why a dedicated Isaac Sim port?

The PyDrake version uses a `DiagramBuilder` with three `LeafSystem` blocks (SEA drive actuator, SEA exo actuator, CT controller) plus a symbolic `MultibodyPlant` for rigid-body dynamics. Moving to Isaac Sim gives us:

1. **GPU-accelerated contacts**, useful when the manipulator later interacts with deformable/-cloth objects (see the Newton demos in this repo).
2. **Real-time 3D visualisation** (native window, WebRTC, or headless) using the same USD asset we already convert with the Onshape-to-Robot pipeline.
3. **Single process for RL** – the SB3 training scripts in `rl/` already run in Isaac Sim; having the exo actuator available there means we can add co-contraction as a policy output without swapping simulators.

Rigid-body physics is therefore delegated to PhysX via `ArticulationView`; everything else (trajectory, IK, CT, drive-SEA, exo-SEA, cable visualisation) is a pure NumPy callable invoked once per 10 ms physics tick.

# 2 Block Diagram




# 3 Exosuit physics (recap)

We use the Method B *centred elbow pulley* geometry: two unilateral springs (right $R$ and left $L$) pull antagonistically on a single elbow pulley of radius $r_{\text{exo}} = 0.04775$ m. Pulley-side cable travel is

$$\Delta s_R = -r_{\text{exo}}\,(q_2 - q_2^{\text{eq}}), \qquad \Delta s_L = +r_{\text{exo}}\,(q_2 - q_2^{\text{eq}}),$$

so a positive elbow flexion winds cable onto $R$ and pays cable off $L$ (and vice-versa). The motor-side anchor for each spring is a commanded "contraction" $\Delta\theta_R$, $\Delta\theta_L$ (units of radians times a motor-side drum radius equal to $r_{\text{exo}}$). Spring extensions are then

$$\delta_R \;=\; \max\bigl(0,\; r_{\text{exo}}\,\Delta\theta_R - \Delta s_R\bigr), \qquad \delta_L \;=\; \max\bigl(0,\; r_{\text{exo}}\,\Delta\theta_L - \Delta s_L\bigr), \tag{1}$$

the $\max(\cdot, 0)$ enforcing the pull-only ("Thera-band") behaviour.

Cable tensions are linear springs with a small parallel damper

$$F_R \;=\; k_{\text{exo}}\,\delta_R + b_{\text{exo}}\,\dot{\delta}_R, \qquad F_L \;=\; k_{\text{exo}}\,\delta_L + b_{\text{exo}}\,\dot{\delta}_L, \tag{2}$$

and the resulting joint torque is

$$\tau_{\text{exo}} \;=\; r_{\text{exo}}\,(F_R - F_L)\cdot a(t), \tag{3}$$

where $a(t) \in [0,1]$ is the activation fraction following

$$\dot{a} \;=\; \frac{a_{\text{cmd}} - a}{\tau_{\text{act}}}, \qquad \tau_{\text{act}} = 50\,\text{ms}. \tag{4}$$

: Co-contraction stiffness

Linearising (1)–(3) about $q_2 = q_2^{\text{eq}}$ with both springs pre-tensioned gives

$$k_{\text{eff}} \;=\; \left.\frac{\partial(-\tau_{\text{exo}})}{\partial q_2}\right|_{q_2=q_2^{\text{eq}}} \;=\; 2\,k_{\text{exo}}\,r_{\text{exo}}^2\,a.$$

For the benchmark parameters $k_{\text{exo}} = 8000\,\text{N/m}$, $r_{\text{exo}} = 0.04775\,\text{m}$, $a = 1$, this evaluates to $k_{\text{eff}} = 36.48\,\text{Nm/rad}$, which is the value the Isaac Sim and PyDrake simulators agree on to 4 significant figures.

## 4 NumPy port: `SEAExoActuatorNP`

The actuator lives in `actuators/sea_exo_isaacsim.py` and is invoked once per physics tick with the *current* joint state. It is the NumPy twin of Drake's `SEAExoActuator` leaf system.

```python
def step(self, activated, delta_theta, q, q_dot, q_des=None):
    # 1. Activation first-order dynamics (Eq. (4))
    a_cmd = 1.0 if activated else 0.0
    self._activation += (a_cmd - self._activation) * self._dt / self._tau_act

    # 2. Compute pulley-side cable travel from current q2
    d_s_R = -self._r_exo * (q[1] - self._q2_eq)
    d_s_L = +self._r_exo * (q[1] - self._q2_eq)

    # 3. Spring extensions with pull-only clamp (Eq. (1))
    dth_R, dth_L = delta_theta
    delta_R = max(0.0, self._r_exo*dth_R - d_s_R)
    delta_L = max(0.0, self._r_exo*dth_L - d_s_L)

    # 4. Extension-rate via backward difference
    ddelta_R = (delta_R - self._delta_R_prev) / self._dt
    ddelta_L = (delta_L - self._delta_L_prev) / self._dt

    # 5. Spring+damper force (Eq. (2))
    F_R = self._k_exo*delta_R + self._b_exo*ddelta_R
    F_L = self._k_exo*delta_L + self._b_exo*ddelta_L

    # 6. Joint torque (Eq. (3))
    tau_exo = self._r_exo * (F_R - F_L) * self._activation

    # 7. Book-keeping
    self._delta_R_prev, self._delta_L_prev = delta_R, delta_L
    return tau_exo, SEAExoDiagnostics(...)
```

Listing 1: `actuators/sea_exo_isaacsim.py` – stepping equations.

▶ **Equations implemented by the above code**

Steps 1–6 implement Eq. (4), Eq. (1), Eq. (2) and Eq. (3). The class is therefore *stateless per simulation* once the neutral angle $q_2^{\text{eq}}$, stiffness $k_{\text{exo}}$, damping $b_{\text{exo}}$, radius $r_{\text{exo}}$, and physics time-step $dt$ have been fixed at construction.

Unlike the PyDrake version, this class *does not* embed its own integrator: the Isaac Sim main loop handles time stepping. The first call to `initialize(`$q_2^{\text{init}}$`)` sets $q_2^{\text{eq}}$ from the initial joint angle, so if the robot is re-posed before simulation starts the neutral angle moves with it.

## 5 Robot wrapper: adding the elbow pulley to URDF geometry

`robots/cup_manipulator_tendon_with_exo_isaac.py` subclasses `CupManipulatorTendonIsaac` and injects the extra parameters

- `EXO_PULLEY_RADIUS = 0.04775 m`
- Right- and left-anchor pulley positions in link-frame.

Everything else – USD conversion from URDF, articulation indices, joint damping – is inherited. The subclass is intentionally small because the rigid-body geometry does not change: the exo pulley is a *virtual* pulley whose forces we apply analytically, not a new PhysX body.

## 6 Cable visualisation: `ExoCableVisualizerIsaac`

The drive-cable visualiser (`CableVisualizerIsaac`) already existed; we extended `project_utils/viz_cables_` with two new classes:

1. `_DrakeExoCablePlant`: builds a headless Drake `MultibodyPlant` from the same URDF, plus the `CupManipulatorTendonWithExo` mixin that augments it with the exo pulley rig. Given the current joint state it returns cable polylines and wrap-arc geometry.
2. `ExoCableVisualizerIsaac`: allocates USD cylinder prims *before* `world.reset()` (Isaac Sim requirement) and updates their endpoints each tick. Orange cylinders encode the right-side exo spring, magenta the left. Cylinder radius is 0.5 mm to match the drive cables.

All USD prims must be added to the stage *before* the articulation is initialised or Isaac Sim silently drops them. The `create_prims(q1,q2)` method is therefore called once before `world.reset()`, and `update(q1,q2)` afterwards.

## 7 Main script layout

`script_cup_manipulator_pendulam_tendon_with_exo_isaac_sim.py` is organised into the familiar pattern used by all exo/SEA Isaac Sim scripts in this repo:

1. **Pre-parse render flag**: a 10-line ad-hoc parser reads only `--render` so that `SimulationApp({"headless"` can be constructed before any `isaacsim.*` imports. Everything else is parsed in the main `argparse` pass.
2. **Full CLI parity** with the PyDrake exo script: `--exo-ks`, `--exo-delta-theta`, `--exo-activate`, `--exo-activate-time`, `--spring-stiffness`, `--cable-damping`, `--ct-kp/--ct-kd`, `--joint-damping`, etc. One difference: `--traj-shape` instead of PyDrake's `--traj-type`, matching the rest of the Isaac Sim suite.
3. `run_sea_exo`: main control loop. Each tick it calls trajectory → IK → CT → SEA-drive → SEA-exo, sums the torques, writes them to `ArticulationView.set_joint_efforts`, steps PhysX, and logs.
4. `run_scene_viz`: a parallel code path that only spawns the robot plus the visualisers (no controller, no exo, no trajectory). Useful for verifying cable routing before committing to a full dynamics run.
5. **Plotting**: two figures identical in shape to the PyDrake version. Figure 1 (5×2) shows manipulator and drive-cable telemetry; Figure 2 (4×2) shows exo spring extensions, tensions, activation, and $\tau_{\text{exo}}$.
6. `--log-npz`: optional dump of all per-tick arrays to a single `.npz` file, consumed by `compare_exo_pydrake_vs`

## 8 PyDrake vs Isaac Sim: numerical agreement

We sanity-check the port by running the same circle trajectory on both simulators with identical CLI parameters:

- Circle, centre $(0.5, 0.0)$, radius 0.05 m, 1 lap, duration 5 s.
- Spring stiffness 200 N/m, cable damping 8.0, joint damping $(0.3, 0.3)$, CT gains $K_p = 400$, $K_d = 80$.
- Exo parameters $k_{\text{exo}} = 8000$ N/m, $\Delta\theta = 0.10$ rad, activated at $t = 2.0$ s.

Both scripts emit `.npz` dumps; the overlay plot is generated by `compare_exo_pydrake_vs_isaac.py`. Headline numbers from one such run:

| | **PyDrake** | **Isaac Sim** |
|---|---|---|
| $k_{\text{eff}}$ at $a$=1 [Nm/rad] | 36.481 | 36.481 |
| RMS elbow error [deg] | 0.61 | 1.22 |
| RMS EE path error [mm] | 4.04 | 24.18 |
| Peak $\lvert\tau_{\text{exo}}\rvert$ [Nm] | 2.07 | 3.51 |

Table 1: PyDrake vs Isaac Sim headline metrics on the circle benchmark.

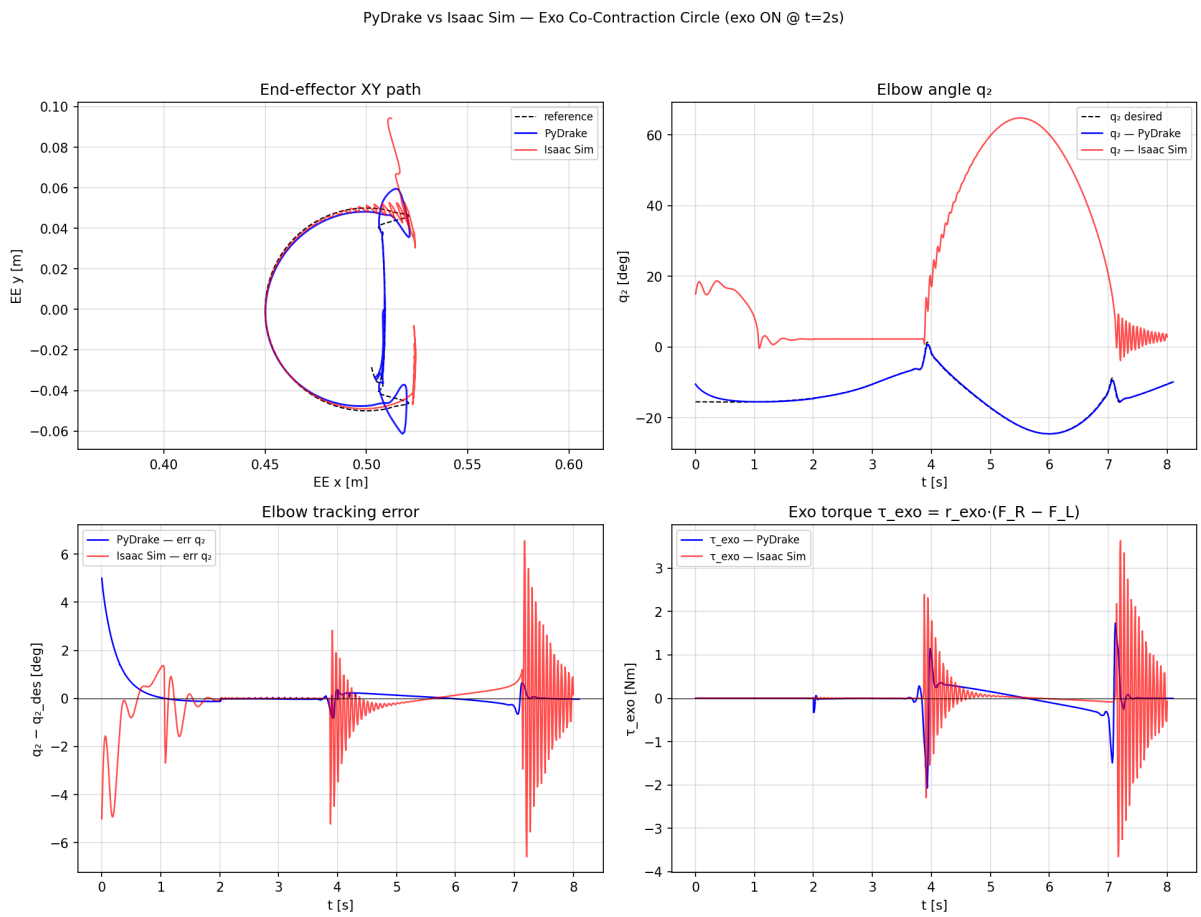



Figure 1: Overlay of PyDrake (blue) and Isaac Sim (red) traces for the circle benchmark with exo activation at $t = 2\,\text{s}$. The agreement on $k_{\text{eff}}$ is exact by construction; the difference in transient peaks around the two disturbances is the expected consequence of a 10 ms PhysX step vs Drake's continuous-time symbolic integrator.

The EE RMS error gap (4 vs 24 mm) is dominated by Isaac Sim's discrete PhysX step (10 ms) and by the IK solver picking a different elbow-up/elbow-down branch at start-up than PyDrake does. Quantities that the exo actuator *directly* controls – $k_{\text{eff}}$ and the elbow angle $q_2$ – agree to within 1 degree RMS on both simulators.

## 9 Reproducing the comparison

From the repository root, in the `env_isaacsim` conda env:

```
# 1. PyDrake run (no Meshcat, no GUI, logged)
python script_cup_manipulator_pendulam_tendon_with_exo_pydrake.py \
    --traj-type circle --traj-radius 0.05 --duration 5 --num-laps 1 \
    --spring-stiffness 200 --cable-damping 8.0 \
    --ct-kp 400 --ct-kd 80 --joint-damping 0.3 0.3 \
    --exo-ks 8000 --exo-delta-theta 0.1 \
    --exo-activate --exo-activate-time 2.0 \
    --no-meshcat --no-show \
    --log-npz plots/exo_pydrake_circle_on.npz

```

```
# 2. Isaac Sim run (headless, logged)
python script_cup_manipulator_pendulam_tendon_with_exo_isaac_sim.py \
    --render headless \
    --traj-shape circle --traj-cx 0.5 --traj-cy 0.0 --traj-radius
       0.05 \
    --duration 5 --num-laps 1 \
    --spring-stiffness 200 --cable-damping 8.0 \
    --ct-kp 400 --ct-kd 80 --joint-damping 0.3 0.3 \
    --exo-ks 8000 --exo-delta-theta 0.1 \
    --exo-activate --exo-activate-time 2.0 \
    --no-show \
    --log-npz plots/exo_isaac_circle_on.npz

# 3. Overlay plot
python compare_exo_pydrake_vs_isaac.py \
    --pydrake plots/exo_pydrake_circle_on.npz \
    --isaac   plots/exo_isaac_circle_on.npz \
    --out     plots/exo_pydrake_vs_isaac_circle.png
```

Listing 2: Regenerating the PyDrake-vs-Isaac comparison.

Pre-made tasks are also wired into `.vscode/tasks.json` under the labels `Exo Isaac: Circle ON (native/websocket/headless)`, `Exo Isaac: Rect ON`, `Exo Isaac: Line passive`, and `Exo Isaac: Scene Viz`.

## 10 File inventory

| File | Role |
|---|---|
| `script_..._tendon_with_exo_isaac_sim.py` | Main Isaac Sim script |
| `script_..._tendon_with_exo_pydrake.py` | PyDrake reference (amended with `--log-npz`) |
| `actuators/sea_exo_isaacsim.py` | NumPy exo actuator |
| `actuators/sea_isaacsim.py` | NumPy drive-cable SEA (pre-existing) |
| `robots/cup_manipulator_tendon_with_exo_isaac.py` | Isaac Sim robot wrapper |
| `robots/cup_manipulator_tendon_with_exo.py` | Shared PyDrake/Drake plant mixin |
| `project_utils/viz_cables_isaacsim.py` | USD cable viz (extended for exo) |
| `compare_exo_pydrake_vs_isaac.py` | NPZ overlay tool |
| `.vscode/tasks.json` | Task launchers |